Student Performance Prediction

Predicting the final grade (G3) of students using the UCI Student Performance dataset.

Dataset source: Kaggle, 'Math Students' (originally from the UCI Machine Learning Repository, Cortez and Silva, 2008)

Step 0: Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
```

Step 1: Load the data

```
df = pd.read_csv("student-mat.csv", sep=",")
print(df.shape)
df.head()
```

```
(395, 33)
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...
1	GP	F	17	U	GT3	T	1	1	at_home	other	...
2	GP	F	15	U	LE3	T	1	1	at_home	other	...
3	GP	F	15	U	GT3	T	4	2	health	services	...
4	GP	F	16	U	GT3	T	3	3	other	other	...

5 rows × 33 columns

Step 2: Check for missing values

```
# count missing values and duplicate rows
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
df.info()
```

```
Missing values: 0
Duplicate rows: 0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64 
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64 
 7   Fedu        395 non-null    int64 
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64 
 13  studytime   395 non-null    int64 
 14  failures    395 non-null    int64 
 15  schoolsup   395 non-null    object
 16  famsup      395 non-null    object
 17  paid        395 non-null    object
 18  activities  395 non-null    object
 19  nursery     395 non-null    object
 20  higher      395 non-null    object
 21  internet    395 non-null    object
 22  romantic    395 non-null    object
 23  famrel      395 non-null    int64 
 24  freetime    395 non-null    int64 
 25  goout       395 non-null    int64 
 26  Dalc        395 non-null    int64 
 27  Walc        395 non-null    int64 
 28  health      395 non-null    int64 
 29  absences    395 non-null    int64 
 30  G1          395 non-null    int64 
 31  G2          395 non-null    int64 
 32  G3          395 non-null    int64 
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
```

Step 3: Explore the data (EDA)

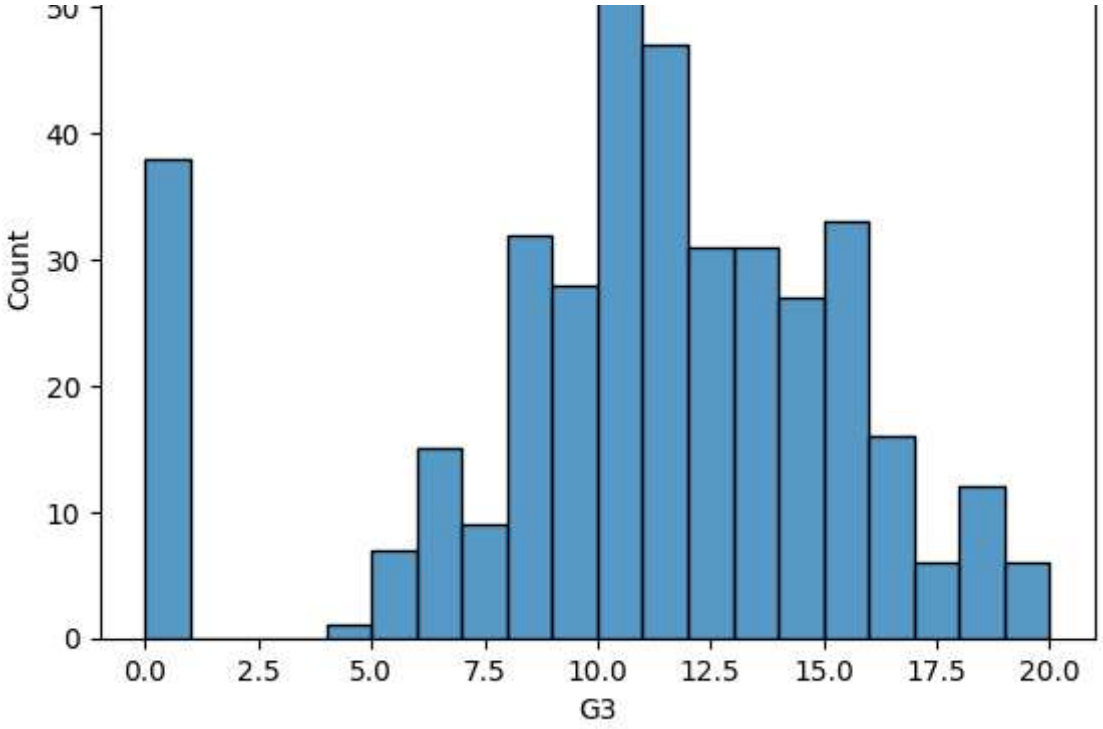
```
# 1. how final grades are spread out
sns.histplot(df["G3"], bins=20)
plt.title("Distribution of final grade (G3)")
```

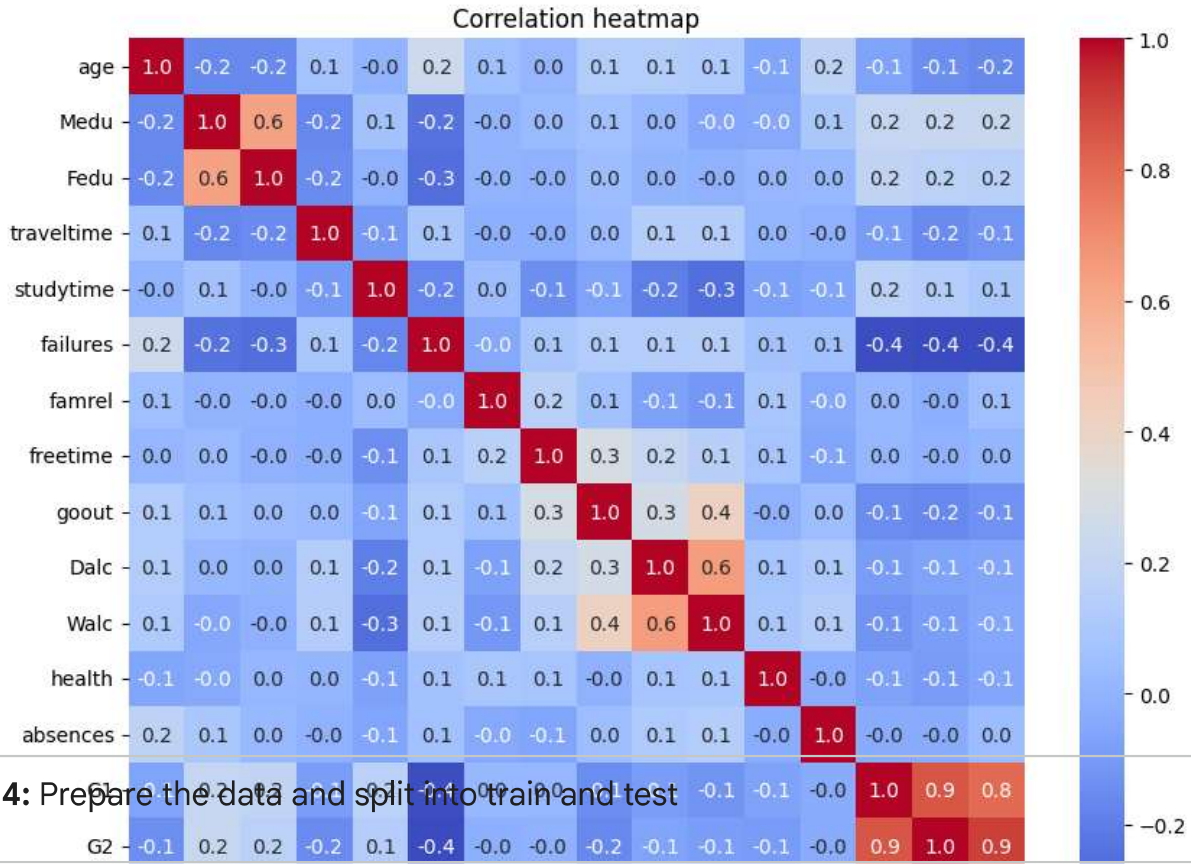
```
plt.show()

# 2. correlation between numeric columns
plt.figure(figsize=(10, 8))
sns.heatmap(df.select_dtypes("number").corr(), annot=True, fmt=".1f", cmap="coo
plt.title("Correlation heatmap")
plt.show()

# 3. past failures vs final grade
sns.boxplot(x="failures", y="G3", data=df)
plt.title("Past failures vs final grade")
plt.show()

# 4. study time vs final grade
sns.boxplot(x="studytime", y="G3", data=df)
plt.title("Study time vs final grade")
plt.show()
```





Step 4: Prepare the data and split into train and test

```
# models only understand numbers, so convert text columns (like "sex", "school"
df_model = pd.get_dummies(df, drop_first=True)
```

```
X = df_model.drop("G3", axis=1)   # inputs
y = df_model["G3"]                # what we want to predict

# keep 20% of students hidden for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
print(X_train.shape, X_test.shape)
```

```
(316, 41) (79, 41)
```

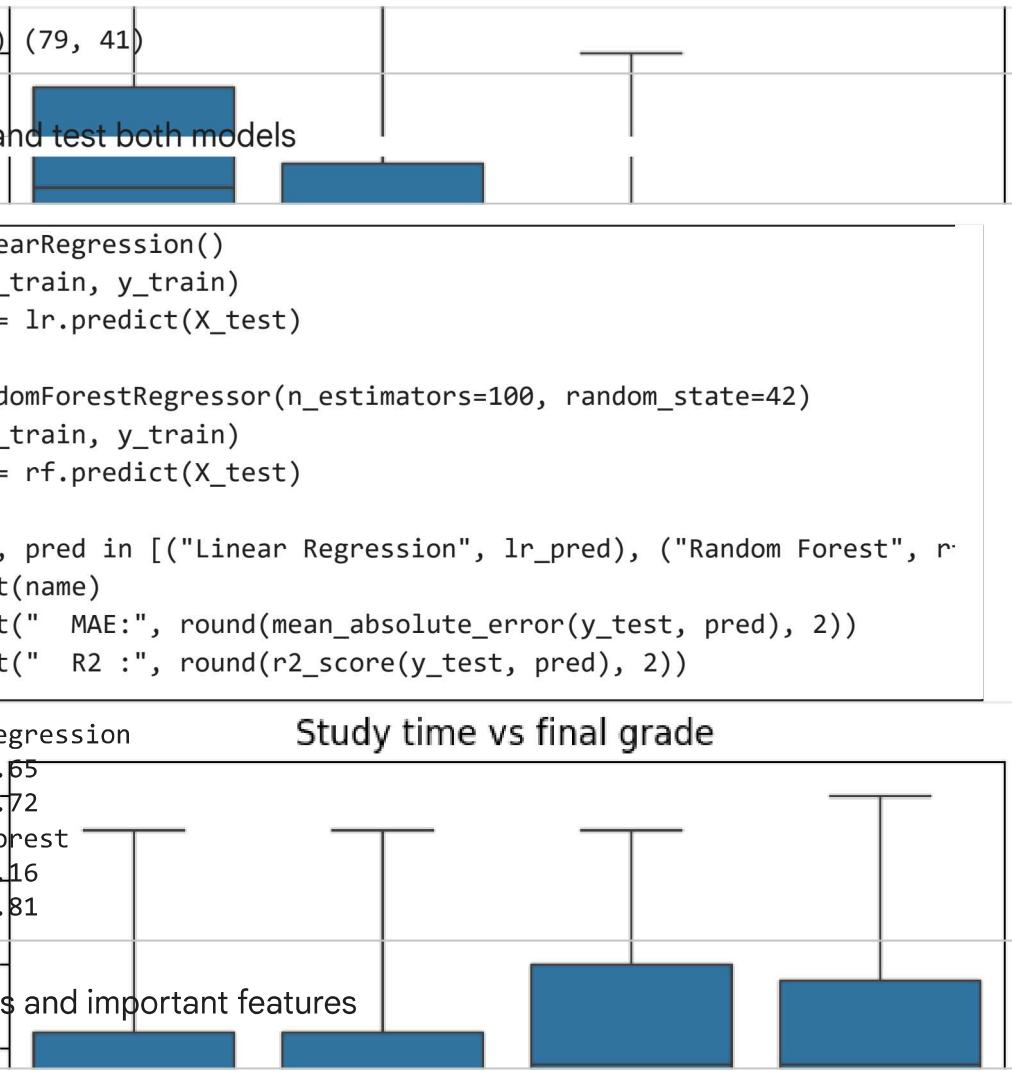
Step 5: Train and test both models

```
lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

for name, pred in [("Linear Regression", lr_pred), ("Random Forest", r
    print(name)
    print("  MAE:", round(mean_absolute_error(y_test, pred), 2))
    print("  R2 :", round(r2_score(y_test, pred), 2))
```

```
Linear Regression
  MAE: 1.65
  R2 : 0.72
Random Forest
  MAE: 1.16
  R2 : 0.81
```



Step 6: Results and important features

```
# which columns mattered most to the Random Forest
importance = pd.Series(rf.feature_importances_, index=X.columns).sort_values()
importance.tail(10).plot(kind="barh")
plt.title("Top 10 important features")
plt.show()

# actual vs predicted grades
plt.scatter(y_test, rf_pred)
plt.plot([0, 20], [0, 20], color="red")
plt.xlabel("Actual grade")
plt.ylabel("Predicted grade")
plt.title("Random Forest: actual vs predicted")
plt.show()
```